МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»

Кафедра информатики и программирования

ИСПОЛЬЗОВАНИЕ ИНДЕКСОВ

ОТЧЕТ

студентки 3 курса 311 группы
направления 02.03.02 — Фундаментальная информатика и информационные технологии
факультета компьютерных наук и информационных технологий
Никитенко Яны Валерьевны

Саратов 2026

1 Код заполнения таблицы

```
-- Заполнение --
-- Клиенты
INSERT INTO Client (Name, LastName, Otchestvo, PhoneNumber, NumberPurchase) VALUES
('Иван', 'Иванов', 'Иванович', '+7(999)123-45-67', 10),
('Мария', 'Петрова', 'Сергеевна', '+7(912)345-67-89', 25),
('Алексей', 'Сидоров', 'Алексеевич', '+7(905)678-90-12', 5),
('Ольга', 'Смирнова', 'Ивановна', '+7(916)234-56-78', 15),
('Дмитрий', 'Кузнецов', 'Дмитриевич', '+7(903)456-78-90', 30),
('Елена', 'Попова', 'Владимировна', '+7(925)567-89-01', 8),
('Сергей', 'Васильев', NULL, '+7(926)678-90-12', 12),
('Анна', 'Павлова', 'Андреевна', '+7(927)789-01-23', 20),
('Андрей', 'Семенов', 'Петрович', '+7(928)890-12-34', 3),
('Наталья', 'Голубева', NULL, '+7(929)901-23-45', 18);
--

-- Старый запрос(он был тестовым, и я решила его оставить)
INSERT INTO Client (Name, LastName, Otchestvo, PhoneNumber, NumberPurchase)
SELECT
    first_names.name,
    last_names.lastname,
    CASE WHEN random() > 0.3 THEN middle_names.middlename ELSE NULL END,
    '+7(9' || lpad((floor(random() * 100))::text, 2, '0') || ')' ||
    lpad((floor(random() * 1000))::text, 3, '0') || '-' ||
    lpad((floor(random() * 100))::text, 2, '0') || '-' ||
    lpad((floor(random() * 100))::text, 2, '0'),
    floor(random() * 50)
FROM
    (VALUES ('Петр'), ('Михаил'), ('Владимир'), ('Екатерина'), ('Светлана'),
            ('Татьяна'), ('Юлия'), ('Александр'), ('Николай'), ('Виктор')) first_names(name)
CROSS JOIN
    (VALUES ('Воробьев'), ('Лебедев'), ('Зайцев'), ('Соловьев'), ('Волков'),
            ('Козлов'), ('Новиков'), ('Морозов'), ('Егоров'), ('Алексеев')) last_names(lastname)
CROSS JOIN
    (VALUES ('Олегович'), ('Михайлович'), ('Владимирович'), ('Николаевич'),
            ('Олеговна'), ('Михайловна'), ('Владимировна'), ('Николаевна')) middle_names(middlename)
LIMIT 40;

-- Цветы
INSERT INTO Flowers (Species, Name, LatName, Country) VALUES
('Роза', 'Красная роза', 'Rosa rubra', 'Нидерланды'),
('Роза', 'Белая роза', 'Rosa alba', 'Нидерланды'),
('Роза', 'Розовая роза', 'Rosa pink', 'Эквадор'),
('Роза', 'Желтая роза', 'Rosa yellow', 'Кения'),
('Тюльпан', 'Тюльпан красный', 'Tulipa gesneriana', 'Россия'),
('Тюльпан', 'Тюльпан желтый', 'Tulipa yellow', 'Россия'),
('Хризантема', 'Хризантема белая', 'Chrysanthemum', 'Италия'),
('Хризантема', 'Хризантема желтая', 'Chrysanthemum', 'Италия'),
```

```
('Пион', 'Пион розовый', 'Paeonia lactiflora', 'Франция'),
('Пион', 'Пион белый', 'Paeonia alba', 'Франция'),
('Гвоздика', 'Гвоздика красная', 'Dianthus', 'Россия'),
('Гортензия', 'Гортензия голубая', 'Hydrangea', 'Япония');
--

-- Букеты
INSERT INTO Bouquet (Name, Quantity, Structure, Price) VALUES
('Романтический', 15, 'Красные розы, зелень', 3500),
('Нежный', 11, 'Белые розы, пионы', 4200),
('Солнечный', 9, 'Желтые тюльпаны, хризантемы', 2800),
('Королевский', 25, 'Розы, пионы, хризантемы', 6500),
('Минимализм', 7, 'Белые хризантемы', 1800),
('Весенний', 13, 'Тюльпаны, нарциссы', 3200),
('Свадебный', 21, 'Белые розы, гортензия', 5500),
('Юбилейный', 31, 'Ассорти из 5 видов цветов', 7800),
('Детский', 9, 'Мелкие разноцветные цветы', 2100),
('Премиум', 51, 'Эксклюзивная композиция', 12500);
--

-- Типы товаров
INSERT INTO TypeGoods (NameType) VALUES
('Букеты'), ('Цветы в упаковке'), ('Горшечные растения'),
('Сухоцветы'), ('Открытки'), ('Мягкие игрушки'),
('Композиции'), ('Сопутствующие товары');
--

-- Прочее
INSERT INTO AnotherGoods (Name) VALUES
('Роза красная в упаковке'), ('Тюльпаны набор 5 шт'), ('Кактус в горшке'),
('Открытка "С любовью"'), ('Медведь 30 см'), ('Лаванда сухая'),
('Хризантема в горшке'), ('Открытка "С днём рождения"'), ('Зайка мягкая'),
('Букет розовый смешанный'), ('Роза белая в упаковке'), ('Фиалка в горшке'),
('Набор открыток'), ('Собачка мягкая'), ('Эустома в горшке'),
('Композиция в корзине'), ('Ваза стеклянная'), ('Лента упаковочная');
--

-- Цветы и букеты. Связь
INSERT INTO FlowersAndBouquet (IdFlower, IdBouquet) VALUES
-- Романтический (букет 1)
(1, 1), (2, 1),
-- Нежный (букет 2)
(2, 2), (9, 2), (10, 2),
-- Солнечный (букет 3)
(4, 3), (5, 3), (6, 3), (8, 3),
-- Королевский (букет 4)
(1, 4), (3, 4), (7, 4), (9, 4),
-- Минимализм (букет 5)
```

```
(7, 5),
-- Весенний (букет 6)
(5, 6), (6, 6),
-- Свадебный (букет 7)
(2, 7), (10, 7), (12, 7),
-- Юбилейный (букет 8)
(1, 8), (3, 8), (7, 8), (9, 8), (11, 8),
-- Детский (букет 9)
(3, 9), (5, 9), (8, 9),
-- Премиум (букет 10)
(1, 10), (3, 10), (9, 10), (12, 10);
--

-- Товары
INSERT INTO Goods (IdAnotherGoods, IdType, IdBouquet) VALUES
-- Тип 1 (Букеты)
(10, 1, 1), (1, 1, 2), (2, 1, 3), (3, 1, 4), (4, 1, 5),
-- Тип 2 (Цветы в упаковке)
(1, 2, NULL), (2, 2, NULL), (11, 2, NULL),
-- Тип 3 (Горшечные растения)
(3, 3, NULL), (7, 3, NULL), (12, 3, NULL), (15, 3, NULL),
-- Тип 4 (Сухоцветы)
(6, 4, NULL),
-- Тип 5 (Открытки)
(4, 5, NULL), (8, 5, NULL), (13, 5, NULL),
-- Тип 6 (Мягкие игрушки)
(5, 6, NULL), (9, 6, NULL), (14, 6, NULL),
-- Тип 7 (Композиции)
(16, 7, NULL),
-- Тип 8 (Сопутствующие товары)
(17, 8, NULL), (18, 8, NULL);
--

-- Чеки
INSERT INTO Account (IdClient, Status, DatePurchase, Summ, ProcentDiscount, SummAll)
SELECT
    c.Id,
    'оплачено',
    CURRENT_DATE - (floor(random() * 180)::int || ' days')::interval,
    1500 + floor(random() * 10000),
    CASE WHEN random() > 0.7 THEN floor(random() * 20) ELSE 0 END,
   1500 + floor(random() * 10000) * (1 - CASE WHEN random() > 0.7 THEN floor(random() * 20) / 100.0 ELSE 0 END)
FROM Client c
CROSS JOIN generate_series(1, 3) -- по 3 чека на каждого клиента
WHERE c.Id <= 20; -- Для первых 20 клиентов
--

-- Добавление еще активных клиентов
```

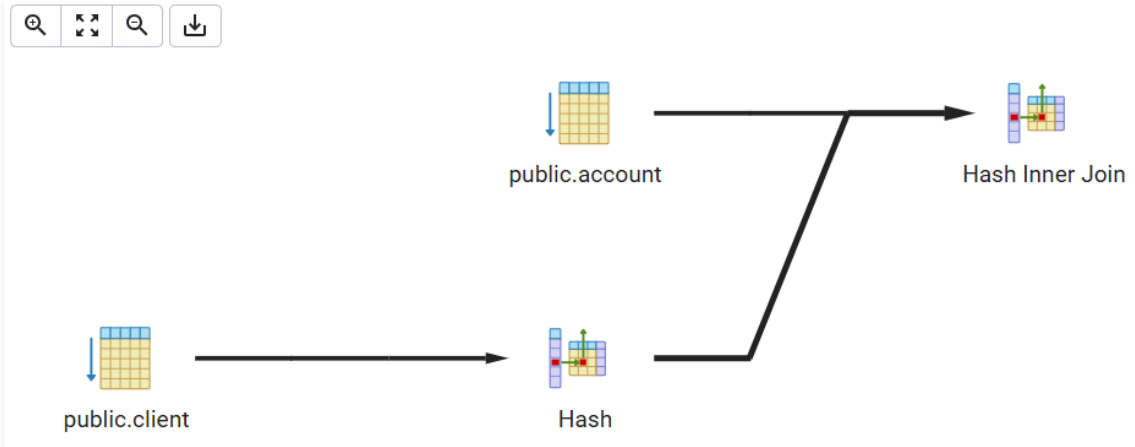
```
INSERT INTO Account (IdClient, Status, DatePurchase, Summ, ProcentDiscount, SummAll)
SELECT
    c.Id,
    CASE WHEN random() > 0.9 THEN 'возврат' ELSE 'оплачено' END,
    CURRENT_DATE - (floor(random() * 60)::int || ' days')::interval,
    2000 + floor(random() * 15000),
    CASE WHEN random() > 0.6 THEN floor(random() * 15) ELSE 0 END,
   2000 + floor(random() * 15000) * (1 - CASE WHEN random() > 0.6 THEN floor(random() * 15) / 100.0 ELSE 0 END)
FROM Client c
CROSS JOIN generate_series(1, 2) -- еще по 2 чека
WHERE c.NumberPurchase > 15; -- Только для клиентов с большим числом покупок

-- Товары в чеке
INSERT INTO GoodsCheck (IdCheck, IdAnotherGoods, IdFlower, Price)
SELECT
    a.Id,
    ag.Id,
    CASE WHEN random() > 0.7 THEN f.Id ELSE NULL END,
    ag.Id * 100 + floor(random() * 1000)
FROM Account a
CROSS JOIN AnotherGoods ag
LEFT JOIN Flowers f ON f.Id = ag.Id % 12 + 1
WHERE random() > 0.7 -- Не для всех товаров, чтобы не было слишком много записей
LIMIT 100;
```

2 B-Tree индексы

2.1 Простой B-Tree индекс

B-tree (сбалансированное дерево) — это самый распространенный тип индекса в PostgreSQL. Он поддерживает все стандартные операции сравнения (>, <, >=, <=, =, <>) и может использоваться с большинством типов данных. B-tree индексы могут быть использованы для сортировки, ограничений уникальности и поиска по диапазону значений. Чаще всего они применяются для ускорения операций соединения таблиц (JOIN).

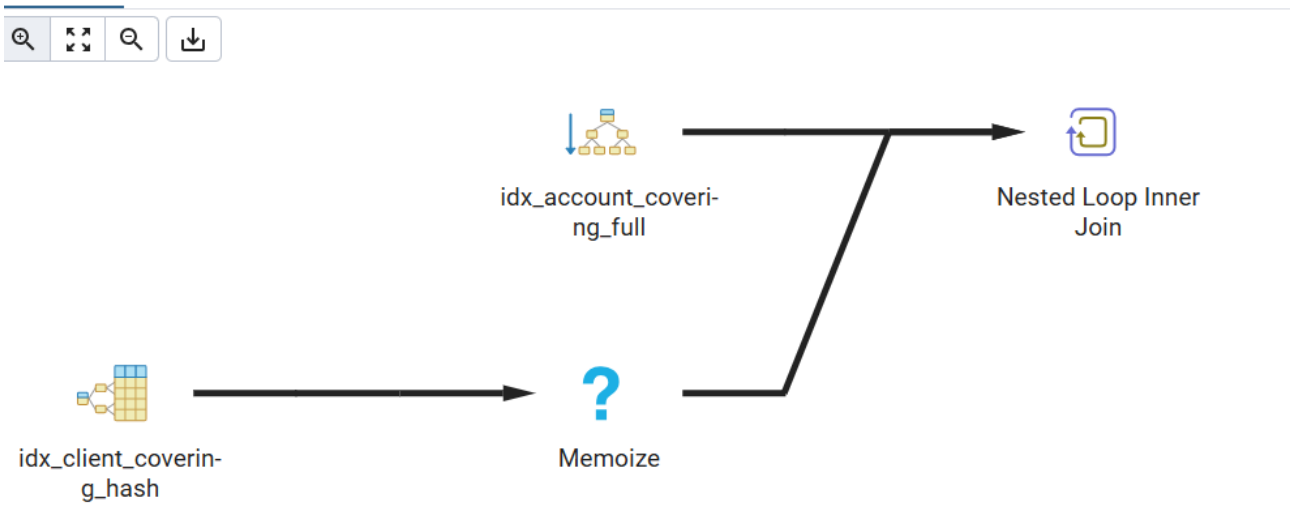
```
SELECT a.Id, a.DatePurchase, a.Summ, c.Name, c.LastName
FROM Account a
INNER JOIN Client c ON a.IdClient = c.Id;
```



Graphical Analysis Statistics

#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Hash Inner Join (cost=4.12..7.75 rows=126 width=36) (actual=2.001... **Hash Cond**: (a.idclient = c.id)	0.087 ms	2.086 ms	↑ 1	126	126	1
2.	→ Seq Scan on public.account as a (cost=0..3.26 rows=126 width...	1.932 ms	1.932 ms	↑ 1	126	126	1
3.	→ Hash (cost=3.5..3.5 rows=50 width=27) (actual=0.066..0.067 ro... **Buckets**: 1024 **Batches**: 1 **Memory Usage**: 12 kB	0.023 ms	0.067 ms	↑ 1	50	50	1
4.	→ Seq Scan on public.client as c (cost=0..3.5 rows=50 width...	0.045 ms	0.045 ms	↑ 1	50	50	1

```
CREATE INDEX idx_account_idclient_btree ON Account(IdClient);
```



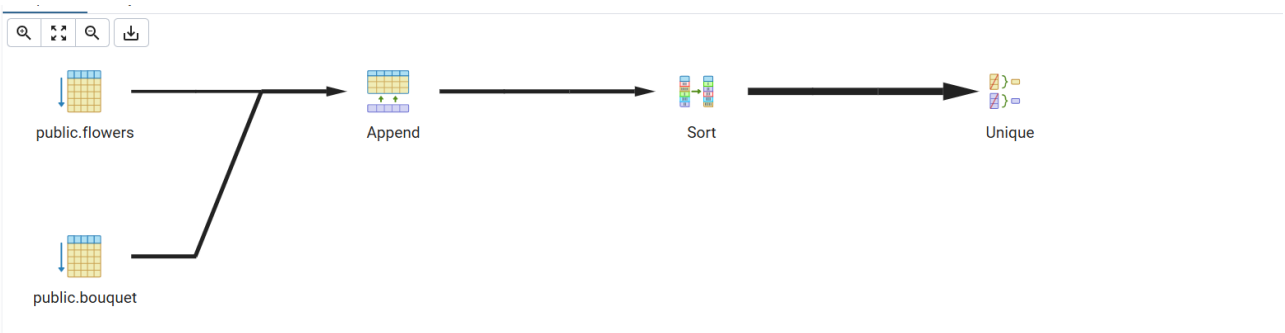
Graphical Analysis Statistics

#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Nested Loop Inner Join (cost=0.15..28.98 rows=126 width=36) (act...	0.096 ms	0.128 ms	↑ 1	126	126	1
2.	→ Index Only Scan using idx_account_covering_full on public.acco...	0.031 ms	0.031 ms	↑ 1	126	126	1
3.	→ Memoize (cost=0.01..0.25 rows=1 width=27) (actual=0.001..0.0... **Buckets**: **Batches**: **Memory Usage**: 6 kB	0.001 ms	0.001 ms	↑ 1	1	1	126
4.	→ Index Scan using idx_client_covering_hash on public.client... **Index Cond**: (c.id = a.idclient)	0.001 ms	0.001 ms	↑ 0.01	1	1	40

2.2 Уникальный B-Tree индекс

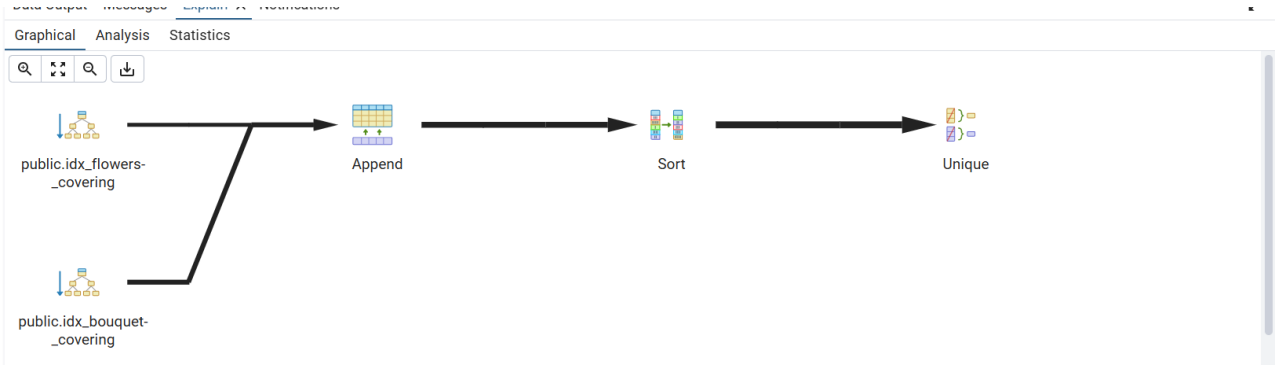
Уникальный B-Tree индекс — это специализированный вариант B-Tree индекса, который гарантирует уникальность данных в индексируемых столбцах. Помимо ускорения поиска, он обеспечивает целостность данных, предотвращая дублирование значений. Такой индекс автоматически создаётся при определении ограничения UNIQUE для таблицы.

SELECT Name AS ItemName, ’Flower’ AS ItemType FROM Flowers
UNION
SELECT Name, ’Bouquet’ FROM Bouquet
ORDER BY ItemName



#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Unique (cost=63.83..70.07 rows=832 width=178) (actual=2.359..2.3...	0.018 ms	2.379 ms	↑ 37.82	22	832	1
2.	→ Sort (cost=63.83..65.91 rows=832 width=178) (actual=2.357..2....	2.328 ms	2.361 ms	↑ 37.82	22	832	1
3.	→ Append (cost=0..23.48 rows=832 width=178) (actual=0.02...	0.004 ms	0.034 ms	↑ 37.82	22	832	1
4.	→ Seq Scan on public.flowers as flowers (cost=0..1.12 r...	0.025 ms	0.025 ms	↑ 1	12	12	1
5.	→ Seq Scan on public.bouquet as bouquet (cost=0..18.2 ...	0.006 ms	0.006 ms	↑ 82	10	820	1

CREATE UNIQUE INDEX idx_client_phone_unique ON Client(PhoneNumber);



Graphical Analysis Statistics

#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Unique (cost=117.11..123.35 rows=832 width=178) (actual=0.102..0...	0.006 ms	0.108 ms	↑ 37.82	22	832	1
2.	→ Sort (cost=117.11..119.19 rows=832 width=178) (actual=0.102....	0.063 ms	0.103 ms	↑ 37.82	22	832	1
3.	→ Append (cost=0.14..76.76 rows=832 width=178) (actual=0...	0.003 ms	0.04 ms	↑ 37.82	22	832	1
4.	→ Index Only Scan using idx_flowers_covering on public....	0.03 ms	0.03 ms	↑ 1	12	12	1
5.	→ Index Only Scan using idx_bouquet_covering on public...	0.008 ms	0.008 ms	↑ 82	10	820	1

2.3 Составной B-Tree индекс

Составной B-Tree индекс — это индекс, построенный на двух или более столбцах таблицы. Он эффективен для запросов, которые фильтруют или сортируют по нескольким полям одновременно. Порядок столбцов в определении индекса критически важен: индекс будет работать для запросов, использующих ведущие (первые) столбцы индекса. В данном случае, индекс оптимален для поиска по дате, а затем по статусу, или только по дате.

– Статистика по дням недели

SELECT

EXTRACT(DOW FROM DatePurchase) AS DayOfWeek,

COUNT(*) AS PurchasesCount,

SUM(Summ) AS TotalSum,

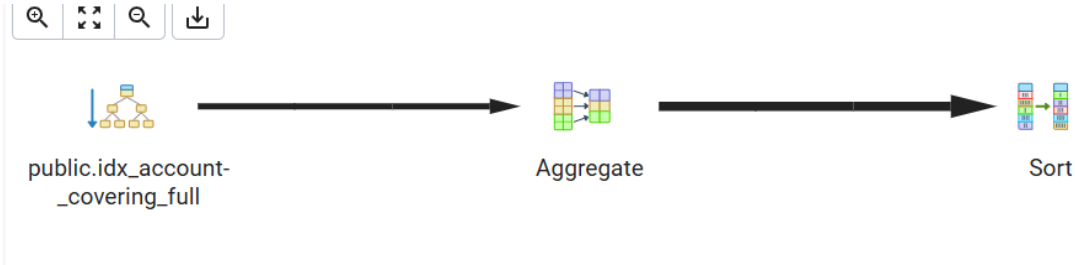
AVG(Summ)::NUMERIC(10,2) AS AvgSum

FROM Account

WHERE DatePurchase IS NOT NULL
GROUP BY EXTRACT(DOW FROM DatePurchase)
ORDER BY DayOfWeek;

#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Sort (cost=8.59..8.79 rows=79 width=88) (actual=3.611..3.612 rows...	0.508 ms	3.612 ms	↑ 11.29	7	79	1
2.	→ Aggregate (cost=4.52..6.1 rows=79 width=88) (actual=3.098..3.... Buckets: Batches: Memory Usage: 24 kB	0.087 ms	3.104 ms	↑ 11.29	7	79	1
3.	→ Seq Scan on public.account as account (cost=0..3.57 rows...	3.018 ms	3.018 ms	↑ 1	126	126	1

CREATE INDEX idx_account_date_status_composite ON Account(DatePurchase, Sta



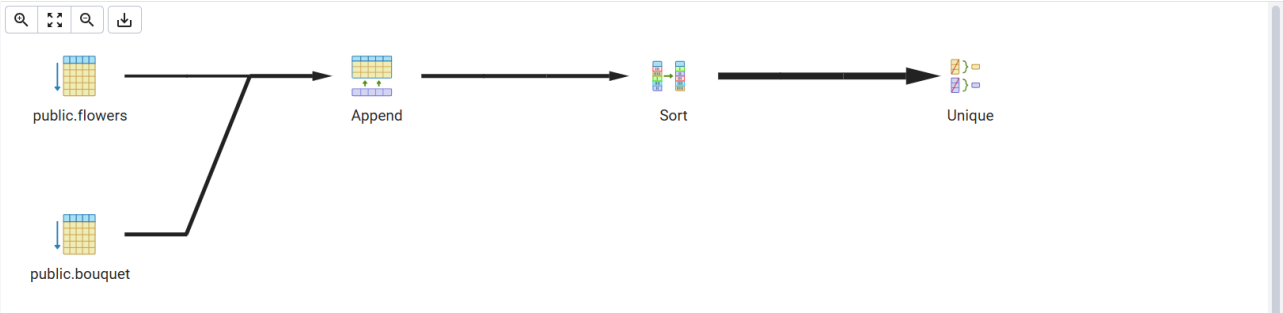
Data Output Messages Explain × Notifications

Graphical Analysis Statistics

#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Sort (cost=21.67..21.87 rows=79 width=88) (actual=0.153..0.154 ro...	0.011 ms	0.154 ms	↑ 11.29	7	79	1
2.	→ Aggregate (cost=17.61..19.18 rows=79 width=88) (actual=0.13... Buckets: Batches: Memory Usage: 24 kB	0.066 ms	0.144 ms	↑ 11.29	7	79	1
3.	→ Index Only Scan using idx_account_covering_full on public....	0.078 ms	0.078 ms	↑ 1	126	126	1

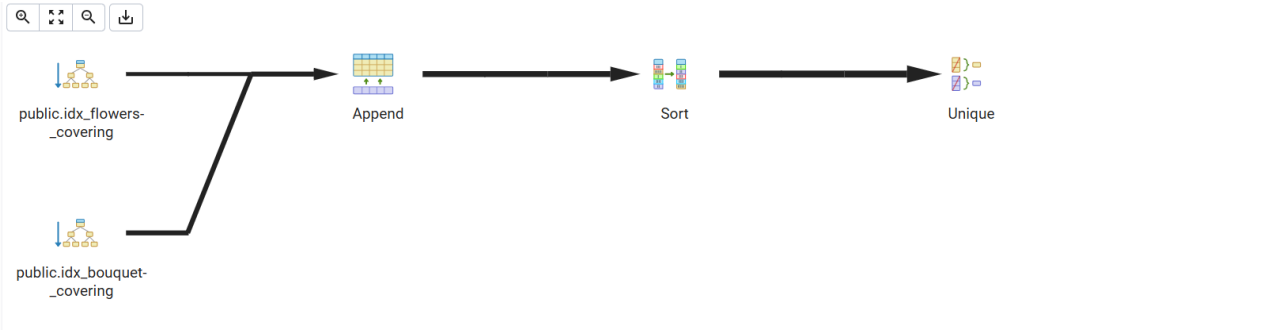
2.4 Индекс с выражениями

SELECT Name AS ItemName, 'Flower' AS ItemType FROM Flowers
UNION
SELECT Name, 'Bouquet' FROM Bouquet
ORDER BY ItemName



#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Unique (cost=63.83..70.07 rows=832 width=178) (actual=0.209..0.2...	0.012 ms	0.221 ms	↑ 37.82	22	832	1
2.	→ Sort (cost=63.83..65.91 rows=832 width=178) (actual=0.208..0....	0.161 ms	0.21 ms	↑ 37.82	22	832	1
3.	→ Append (cost=0..23.48 rows=832 width=178) (actual=0.02...	0.004 ms	0.049 ms	↑ 37.82	22	832	1
4.	→ Seq Scan on public.flowers as flowers (cost=0..1.12 r...	0.029 ms	0.029 ms	↑ 1	12	12	1
5.	→ Seq Scan on public.bouquet as bouquet (cost=0..18.2 ...	0.016 ms	0.016 ms	↑ 82	10	820	1

CREATE INDEX idx_flowers_name_upper ON Flowers(UPPER(Name));



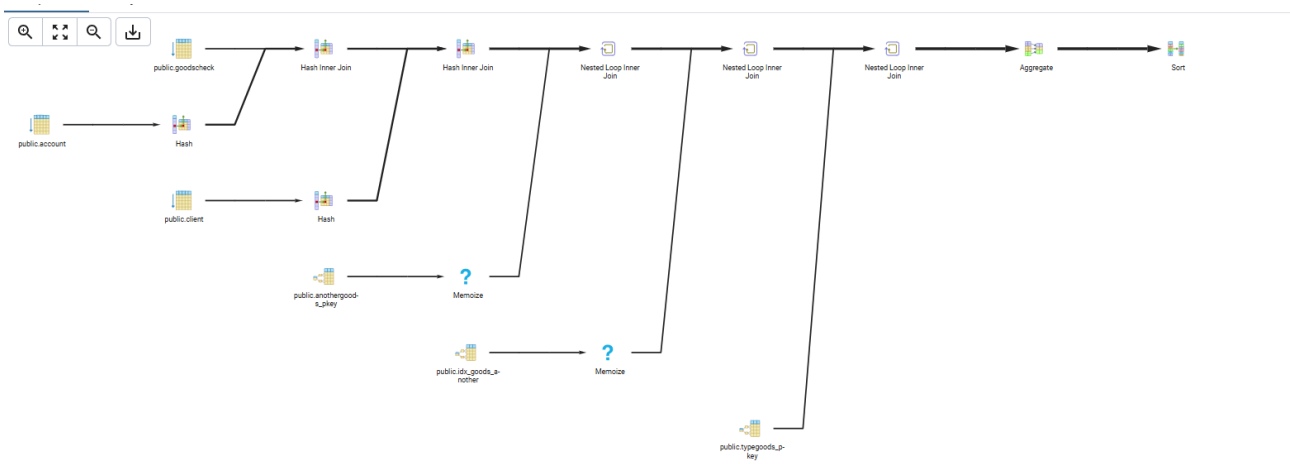
Graphical Analysis Statistics

#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Unique (cost=117.11..123.35 rows=832 width=178) (actual=0.071..0...	0.005 ms	0.076 ms	↑ 37.82	22	832	1
2.	→ Sort (cost=117.11..119.19 rows=832 width=178) (actual=0.07.....	0.05 ms	0.072 ms	↑ 37.82	22	832	1
3.	→ Append (cost=0.14..76.76 rows=832 width=178) (actual=0...	0.002 ms	0.022 ms	↑ 37.82	22	832	1
4.	→ Index Only Scan using idx_flowers_covering on public....	0.015 ms	0.015 ms	↑ 1	12	12	1
5.	→ Index Only Scan using idx_bouquet_covering on public...	0.005 ms	0.005 ms	↑ 82	10	820	1

2.5 Покрывающий индекс

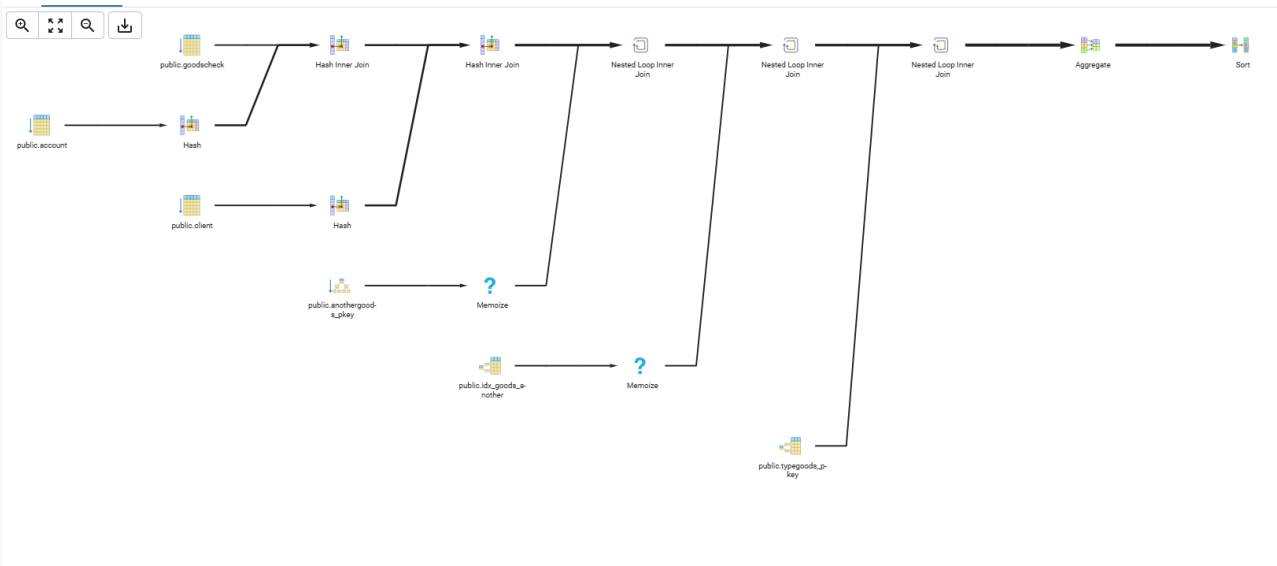
Покрывающий индекс — это индекс, который содержит все столбцы, необходимые для выполнения конкретного запроса. Это позволяет выполнить операцию Index Only Scan: данные для ответа извлекаются непосредственно из индексной структуры, без обращения к основной таблице (heap), что дает максимальную скорость. Ключевое слово INCLUDE добавляет в индекс столбцы, которые не участвуют в поиске, но нужны для выборки.

```
SELECT
    COALESCE(c.Name, 'Все клиенты') AS ClientName,
    COALESCE(tg.NameType, 'Все типы') AS GoodsType,
    COUNT(*) AS ItemsSold,
    SUM(gc.Price) AS TotalRevenue
FROM GoodsCheck gc
JOIN Account a ON gc.IdCheck = a.Id
JOIN Client c ON a.IdClient = c.Id
JOIN AnotherGoods ag ON gc.IdAnotherGoods = ag.Id
JOIN Goods g ON ag.Id = g.IdAnotherGoods
JOIN TypeGoods tg ON g.IdType = tg.IdTypeGoods
GROUP BY GROUPING SETS (
    (c.Name), -- по клиентам
    (tg.NameType), -- по типам товаров
    (c.Name, tg.NameType), -- по комбинации
    () -- общий итог
)
```



1.	→ Sort (cost=77.81..78.66 rows=338 width=138) (actual=2.177..2.188 row...	0.49 ms	2.188 ms	↑ 6.9	49	338	1
2.	→ Aggregate (cost=9.44..63.62 rows=338 width=138) (actual=1.647..... **Buckets**: **Batches**: **Memory Usage**: 96 kB	0.504 ms	1.699 ms	↑ 6.9	49	338	1
3.	→ Nested Loop Inner Join (cost=9.44..54.5 rows=163 width=39) ...	0.504 ms	1.196 ms	↓ 1.23	200	163	1
4.	→ Nested Loop Inner Join (cost=9.29..22.96 rows=163 widt...	0.191 ms	0.69 ms	↓ 1.23	200	163	1
5.	→ Nested Loop Inner Join (cost=9.12..17.24 rows=100 ...	0.139 ms	0.498 ms	↑ 1	100	100	1
6.	→ Hash Inner Join (cost=8.96..11.52 rows=100 wi... **Hash Cond**: (a.idclient = c.id)	0.064 ms	0.358 ms	↑ 1	100	100	1
7.	→ Hash Inner Join (cost=4.83..7.11 rows=100... **Hash Cond**: (gc.idcheck = a.id)	0.089 ms	0.213 ms	↑ 1	100	100	1
8.	→ Seq Scan on public.goodscheck as gc ...	0.029 ms	0.029 ms	↑ 1	100	100	1
9.	→ Hash (cost=3.26..3.26 rows=126 width... **Buckets**: 1024 **Batches**: 1 **Memory Usage**: 13	0.037 ms	0.095 ms	↑ 1	126	126	1
10.	→ Seq Scan on public.account as a (...	0.058 ms	0.058 ms	↑ 1	126	126	1
11.	→ Hash (cost=3.5..3.5 rows=50 width=13) (ac... **Buckets**: 1024 **Batches**: 1 **Memory Usage**: 11 kB	0.025 ms	0.081 ms	↑ 1	50	50	1
13.	→ Memoize (cost=0.16..0.66 rows=1 width=4) (act... **Buckets**: **Batches**: **Memory Usage**: 1 kB	0.001 ms	0.001 ms	↑ 1	1	1	100
14.	→ Index Scan using anothergoods_pkey on pu... **Index Cond**: (ag.id = gc.idanothergoods)	0.001 ms	0.001 ms	↑ 0.01	1	1	4

CREATE INDEX idx_typegoods_covering ON TypeGoods(IdTypeGoods, NameType) INCLUDE ();

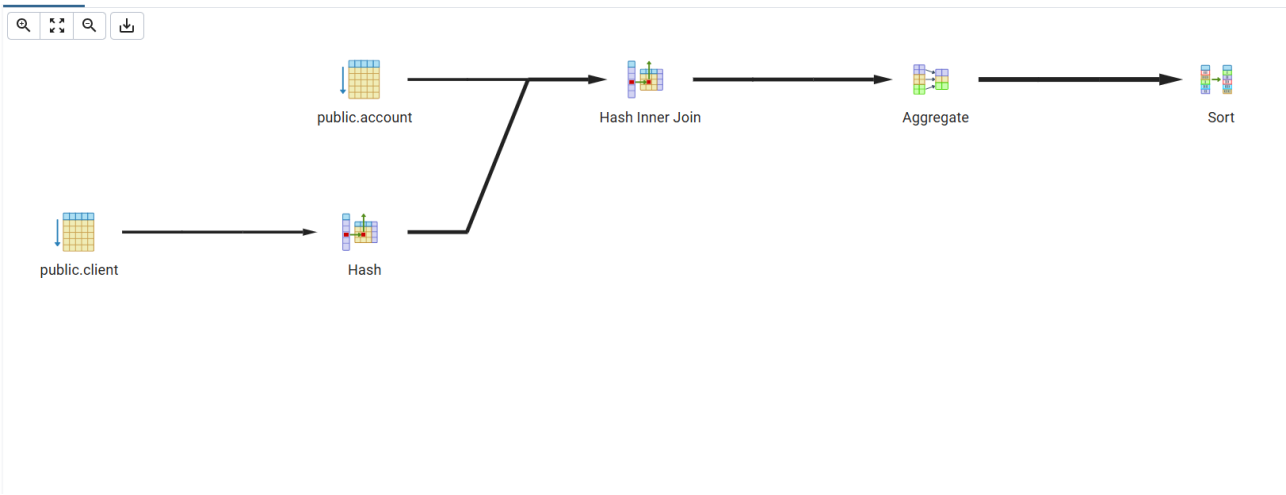


		Exclusive	Inclusive	Rows X	Actual	Plan	Loops
1.	→ Sort (cost=77.81..78.66 rows=338 width=138) (actual=0.956..0.961 row...	0.145 ms	0.961 ms	↑ 6.9	49	338	1
2.	→ Aggregate (cost=9.44..63.62 rows=338 width=138) (actual=0.793..... **Buckets**: **Batches**: **Memory Usage**: 96 kB	0.239 ms	0.817 ms	↑ 6.9	49	338	1
3.	→ Nested Loop Inner Join (cost=9.44..54.5 rows=163 width=39) ...	0.253 ms	0.578 ms	↓ 1.23	200	163	1
4.	→ Nested Loop Inner Join (cost=9.29..22.96 rows=163 widt...	0.084 ms	0.324 ms	↓ 1.23	200	163	1
5.	→ Nested Loop Inner Join (cost=9.12..17.24 rows=100 ...	0.063 ms	0.241 ms	↑ 1	100	100	1
6.	→ Hash Inner Join (cost=8.96..11.52 rows=100 wi... **Hash Cond**: (a.idclient = c.id)	0.031 ms	0.178 ms	↑ 1	100	100	1
7.	→ Hash Inner Join (cost=4.83..7.11 rows=100... **Hash Cond**: (gc.idcheck = a.id)	0.041 ms	0.095 ms	↑ 1	100	100	1
8.	→ Seq Scan on public.goodscheck as gc ...	0.014 ms	0.014 ms	↑ 1	100	100	1
9.	→ Hash (cost=3.26..3.26 rows=126 width... **Buckets**: 1024 **Batches**: 1 **Memory Usage**: 13	0.019 ms	0.04 ms	↑ 1	126	126	1
10.	→ Seq Scan on public.account as a (...	0.022 ms	0.022 ms	↑ 1	126	126	1
11.	→ Hash (cost=3.5..3.5 rows=50 width=13) (ac... **Buckets**: 1024 **Batches**: 1 **Memory Usage**: 11 kB	0.016 ms	0.052 ms	↑ 1	50	50	1
13.	→ Memoize (cost=0.16..0.66 rows=1 width=4) (act... **Buckets**: **Batches**: **Memory Usage**: 1 kB	0	0	↑ 1	1	1	100
14.	→ Index Only Scan using anothergoods_pkey ... **Index Cond**: (ag.id = gc.idanothergoods)	0.001 ms	0.001 ms	↑ 0.01	1	1	4

2.6 Частичный индекс

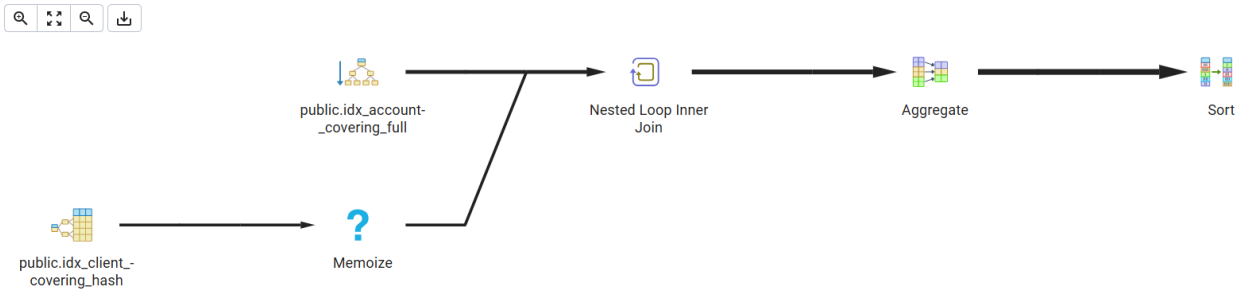
Частичный индекс — это индекс, построенный только для подмножества строк таблицы, которые удовлетворяют заданному условию (WHERE). Он меньше по размеру и часто эффективнее для запросов, соответствующих этому условию, так как индексирует только интересующую нас часть таблицы.

```
  SELECT
    c.Name || ' ' || c.LastName AS FullName,
    COUNT(*) AS TotalPurchases,
    COUNT(*) FILTER (WHERE a.Summ < 2000) AS SmallPurchases,
   COUNT(*) FILTER (WHERE a.Summ BETWEEN 2000 AND 5000) AS MediumPurch
    COUNT(*) FILTER (WHERE a.Summ > 5000) AS LargePurchases,
    SUM(a.Summ) FILTER (WHERE a.Status = 'возврат') AS ReturnedSum,
    SUM(a.Summ) FILTER (WHERE a.Status = 'оплачено') AS PaidSum,
   AVG(a.Summ) FILTER (WHERE a.ProcentDiscount > 0)::NUMERIC(10,2) AS AvgCl
FROM Client c
JOIN Account a ON c.Id = a.IdClient
GROUP BY c.Id, c.Name, c.LastName
HAVING COUNT(*) > 0
ORDER BY TotalPurchases DESC
LIMIT 15;
```



#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Sort (cost=13.95..13.99 rows=17 width=171) (actual=0.696..0.704 r...	0.062 ms	0.704 ms	↓ 2.36	40	17	1
2.	→ Aggregate (cost=12.47..13.6 rows=17 width=171) (actual=0.51... **Filter**: (count(*) > 0) **Rows Removed by Filter**: 0 **Buckets**: **Batches**: **Memory Usage**: 48 kB	0.412 ms	0.642 ms	↓ 2.36	40	17	1
3.	→ Hash Inner Join (cost=4.12..7.75 rows=126 width=51) (act... **Hash Cond**: (a.idclient = c.id)	0.129 ms	0.231 ms	↑ 1	126	126	1
4.	→ Seq Scan on public.account as a (cost=0..3.26 rows=...	0.05 ms	0.05 ms	↑ 1	126	126	1
5.	→ Hash (cost=3.5..3.5 rows=50 width=27) (actual=0.051... **Buckets**: 1024 **Batches**: 1 **Memory Usage**: 12 kB	0.02 ms	0.052 ms	↑ 1	50	50	1
6.	→ Seq Scan on public.client as c (cost=0..3.5 rows=...	0.032 ms	0.032 ms	↑ 1	50	50	1

CREATE INDEX idx_client_vip_partial ON Client(Id, Name, LastName, NumberPurc
WHERE NumberPurchase > 25;

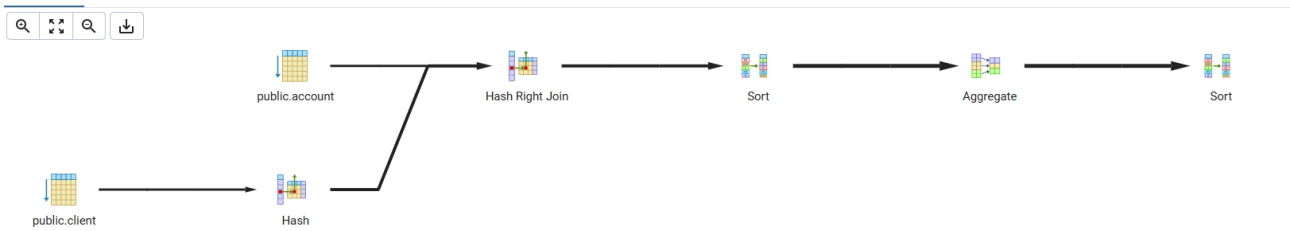


#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Sort (cost=35.18..35.22 rows=17 width=171) (actual=0.341..0.343 r...	0.023 ms	0.343 ms	↓ 2.36	40	17	1
2.	→ Aggregate (cost=33.7..34.83 rows=17 width=171) (actual=0.28... **Filter**: (count(*) > 0) **Rows Removed by Filter**: 0 **Buckets**: **Batches**: **Memory Usage**: 48 kB	0.146 ms	0.321 ms	↓ 2.36	40	17	1
3.	→ Nested Loop Inner Join (cost=0.15..28.98 rows=126 width...	0.121 ms	0.176 ms	↑ 1	126	126	1
4.	→ Index Only Scan using idx_account_covering_full on p...	0.054 ms	0.054 ms	↑ 1	126	126	1
5.	→ Memoize (cost=0.01..0.25 rows=1 width=27) (actual=... **Buckets**: **Batches**: **Memory Usage**: 6 kB	0.001 ms	0.001 ms	↑ 1	1	1	126
6.	→ Index Scan using idx_client_covering_hash on pu... **Index Cond**: (c.id = a.idclient)	0.001 ms	0.001 ms	↑ 0.01	1	1	40

2.7 Частичный покрывающий индекс

Это комбинация частичного индекса и покрывающего индекса (с использованием ключевого слова INCLUDE). Он создает небольшой и эффективный индекс только для определенного подмножества строк (например, для самых ценных клиентов), который при этом содержит все данные для запроса, исключая обращение к таблице.

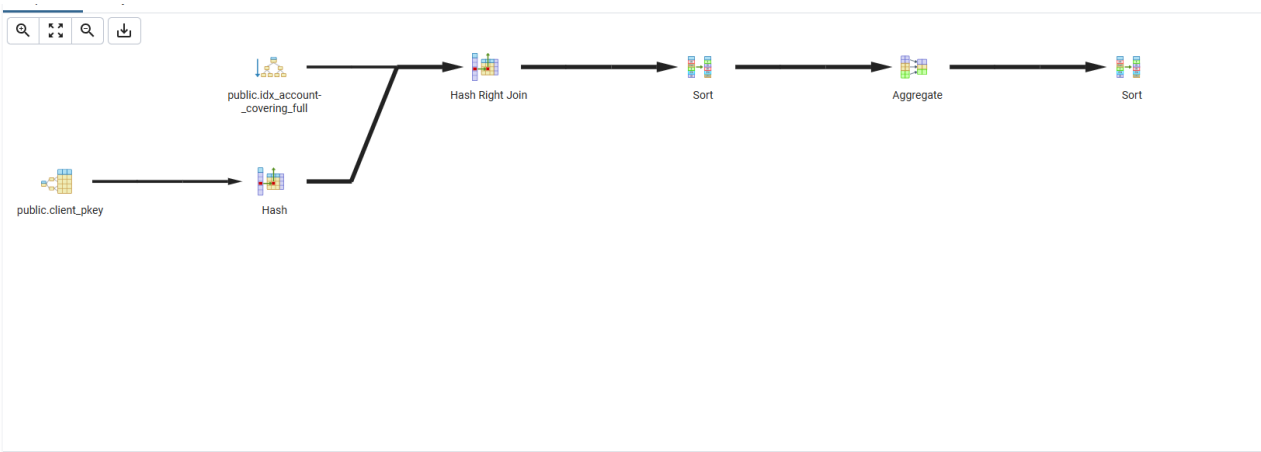
```
SELECT
    CASE
        WHEN c.NumberPurchase < 10 THEN 'Новичок'
        WHEN c.NumberPurchase < 20 THEN 'Регулярный'
        WHEN c.NumberPurchase < 30 THEN 'Постоянный'
        ELSE 'VIP'
    END AS ClientCategory,
    COUNT(DISTINCT c.Id) AS ClientCount,
    COUNT(a.Id) AS TotalPurchases,
    SUM(a.Summ) AS TotalSpent,
    AVG(a.Summ)::NUMERIC(10,2) AS AvgCheck,
   SUM(a.Summ) / NULLIF(COUNT(DISTINCT c.Id), 0)::NUMERIC(10,2) AS SpentPe
FROM Client c
LEFT JOIN Account a ON c.Id = a.IdClient
GROUP BY
    CASE
        WHEN c.NumberPurchase < 10 THEN 'Новичок'
        WHEN c.NumberPurchase < 20 THEN 'Регулярный'
        WHEN c.NumberPurchase < 30 THEN 'Постоянный'
        ELSE 'VIP'
    END
ORDER BY
    MIN(c.NumberPurchase);
```



Graphical Analysis Statistics

#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Sort (cost=17.1..17.19 rows=35 width=132) (actual=0.591..0.593 ro...	0.011 ms	0.593 ms	↑ 8.75	4	35	1
2.	→ Aggregate (cost=13.09..16.2 rows=35 width=132) (actual=0.51...	0.111 ms	0.583 ms	↑ 8.75	4	35	1
3.	→ Sort (cost=13.09..13.4 rows=126 width=49) (actual=0.459....	0.267 ms	0.472 ms	↓ 1.08	136	126	1
4.	→ Hash Right Join (cost=4.12..8.69 rows=126 width=49)... **Hash Cond**: (a.idclient = c.id)	0.113 ms	0.205 ms	↓ 1.08	136	126	1
5.	→ Seq Scan on public.account as a (cost=0..3.26 ro...	0.03 ms	0.03 ms	↑ 1	126	126	1
6.	→ Hash (cost=3.5..3.5 rows=50 width=8) (actual=0.... **Buckets**: 1024 **Batches**: 1 **Memory Usage**: 10 kB	0.017 ms	0.062 ms	↑ 1	50	50	1
7.	→ Seq Scan on public.client as c (cost=0..3.5 r...	0.045 ms	0.045 ms	↑ 1	50	50	1

CREATE INDEX idx_client_vip_covering_partial ON Client(Id, Name, LastName, Nu
INCLUDE (Otchestvo, PhoneNumber)
WHERE NumberPurchase > 25;

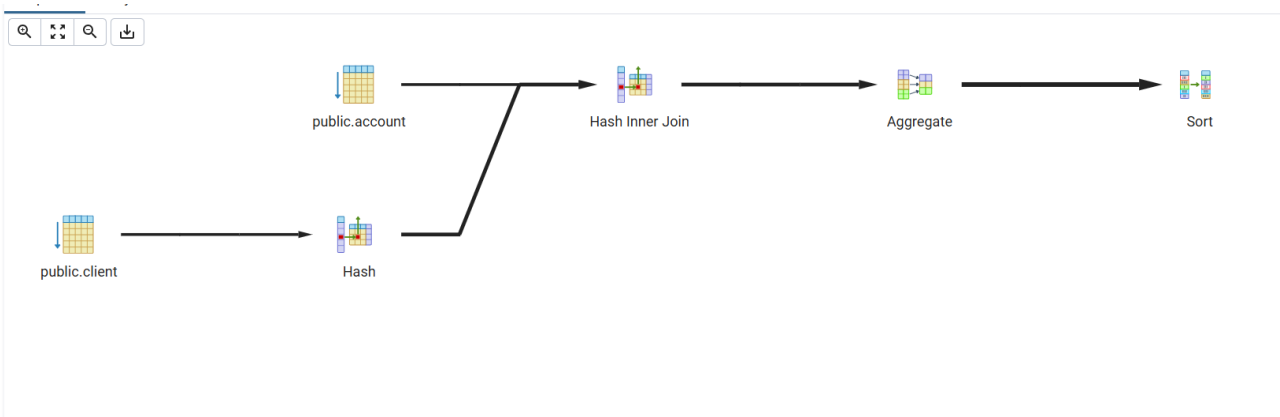


#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Sort (cost=47.29..47.38 rows=35 width=132) (actual=0.394..0.395 r...	0.016 ms	0.395 ms	↑ 8.75	4	35	1
2.	→ Aggregate (cost=43.28..46.39 rows=35 width=132) (actual=0.3...	0.059 ms	0.38 ms	↑ 8.75	4	35	1
3.	→ Sort (cost=43.28..43.59 rows=126 width=49) (actual=0.31...	0.171 ms	0.321 ms	↓ 1.08	136	126	1
4.	→ Hash Right Join (cost=21.37..38.88 rows=126 width=... **Hash Cond**: (a.idclient = c.id)	0.052 ms	0.15 ms	↓ 1.08	136	126	1
5.	→ Index Only Scan using idx_account_covering_full ...	0.05 ms	0.05 ms	↑ 1	126	126	1
6.	→ Hash (cost=20.6..20.6 rows=50 width=8) (actual... **Buckets**: 1024 **Batches**: 1 **Memory Usage**: 10 kB	0.014 ms	0.048 ms	↑ 1	50	50	1
7.	→ Index Scan using client_pkey on public.client...	0.034 ms	0.034 ms	↑ 1	50	50	1

3 Hash индексы

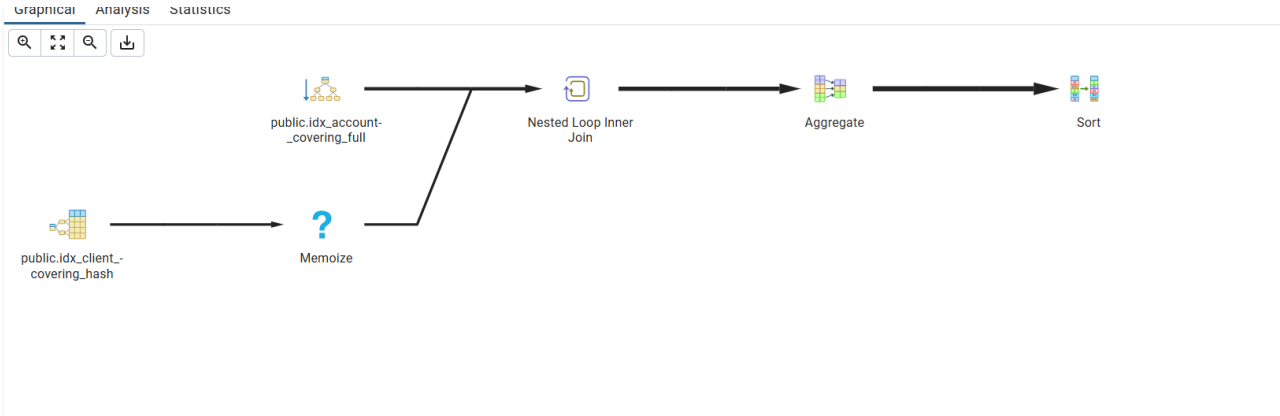
3.1 Простой Hash индекс

```
SELECT
    c.Name || ' ' || c.LastName AS FullName,
    COUNT(*) AS TotalPurchases,
    COUNT(*) FILTER (WHERE a.Summ < 2000) AS SmallPurchases,
   COUNT(*) FILTER (WHERE a.Summ BETWEEN 2000 AND 5000) AS MediumPurch
    COUNT(*) FILTER (WHERE a.Summ > 5000) AS LargePurchases,
    SUM(a.Summ) FILTER (WHERE a.Status = 'возврат') AS ReturnedSum,
    SUM(a.Summ) FILTER (WHERE a.Status = 'оплачено') AS PaidSum,
   AVG(a.Summ) FILTER (WHERE a.ProcentDiscount > 0)::NUMERIC(10,2) AS AvgCl
FROM Client c
JOIN Account a ON c.Id = a.IdClient
GROUP BY c.Id, c.Name, c.LastName
HAVING COUNT(*) > 0
ORDER BY TotalPurchases DESC;
```



#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Sort (cost=13.95..13.99 rows=17 width=171) (actual=0.582..0.587 r...	0.042 ms	0.587 ms	↓ 2.36	40	17	1
2.	→ Aggregate (cost=12.47..13.6 rows=17 width=171) (actual=0.45... Filter: (count(*) > 0) Rows Removed by Filter: 0 Buckets: Batches: Memory Usage: 48 kB	0.32 ms	0.545 ms	↓ 2.36	40	17	1
3.	→ Hash Inner Join (cost=4.12..7.75 rows=126 width=51) (act... Hash Cond: (a.idclient = c.id)	0.093 ms	0.226 ms	↑ 1	126	126	1
4.	→ Seq Scan on public.account as a (cost=0..3.26 rows=...	0.081 ms	0.081 ms	↑ 1	126	126	1
5.	→ Hash (cost=3.5..3.5 rows=50 width=27) (actual=0.052... Buckets: 1024 Batches: 1 Memory Usage: 12 kB	0.023 ms	0.053 ms	↑ 1	50	50	1
6	→ Seq Scan on public.client as c (cost=0..3.5 rows=	0.03 ms	0.03 ms	↑ 1	50	50	1

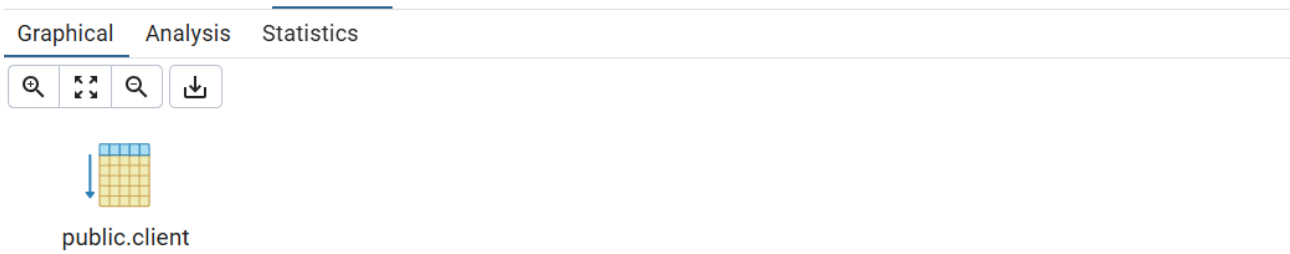
```
CREATE INDEX idx_account_status_hash ON Account USING HASH(Status);
```



#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Sort (cost=35.18..35.22 rows=17 width=171) (actual=0.353..0.356 r...	0.022 ms	0.356 ms	↓ 2.36	40	17	1
2.	→ Aggregate (cost=33.7..34.83 rows=17 width=171) (actual=0.29... **Filter**: (count(*) > 0) **Rows Removed by Filter**: 0 **Buckets**: **Batches**: **Memory Usage**: 48 kB	0.155 ms	0.334 ms	↓ 2.36	40	17	1
3.	→ Nested Loop Inner Join (cost=0.15..28.98 rows=126 width...	0.128 ms	0.18 ms	↑ 1	126	126	1
4.	→ Index Only Scan using idx_account_covering_full on p...	0.051 ms	0.051 ms	↑ 1	126	126	1
5.	→ Memoize (cost=0.01..0.25 rows=1 width=27) (actual=... **Buckets**: **Batches**: **Memory Usage**: 6 kB	0.001 ms	0.001 ms	↑ 1	1	1	126
6.	→ Index Scan using idx_client_covering_hash on pu... **Index Cond**: (c.id = a.idclient)	0.001 ms	0.001 ms	↑ 0.01	1	1	40

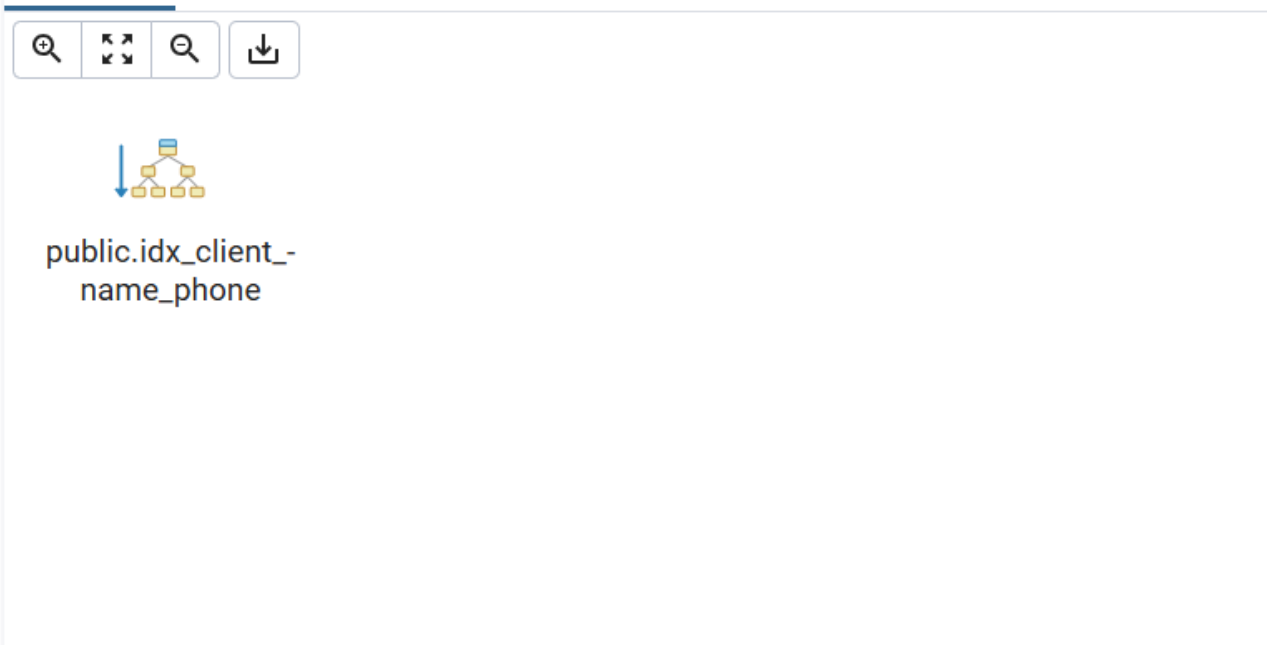
3.2 Hash индекс с использованием выражений

SELECT Name, LastName, PhoneNumber
FROM Client
WHERE Name IN ('Иван', 'Мария', 'Алексей', 'Елена');



#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Seq Scan on public.client as client (cost=0..3.75 rows=4 width=40) (... **Filter**: ((client.name)::text = ANY ('{Иван,Мария,Алексей,Елена}'::text[])) **Rows Removed by Filter**: 46	0.149 ms	0.149 ms	↑ 1	4	4	1

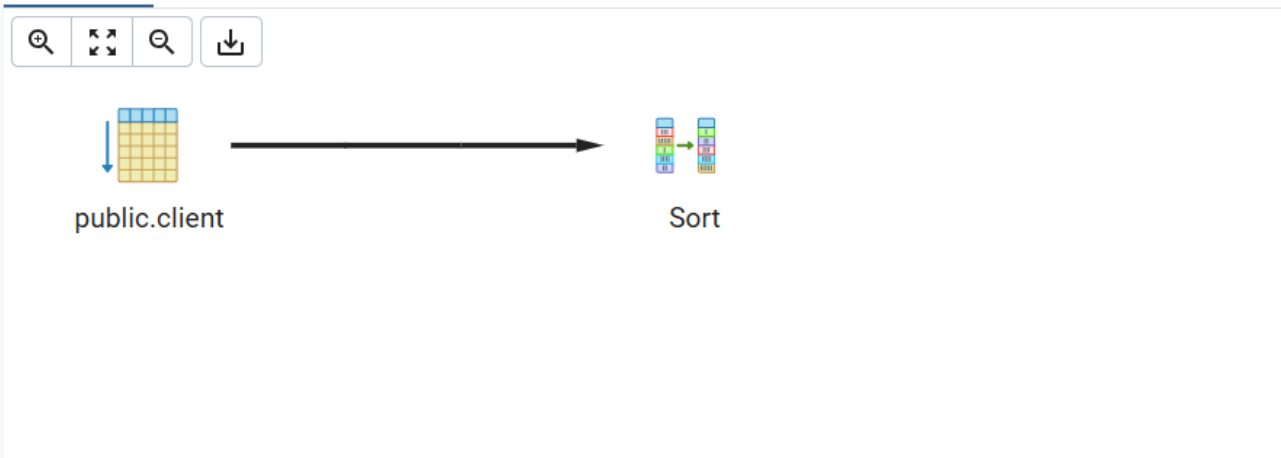
CREATE INDEX idx_client_name_hash ON Client USING HASH(Name);



#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Index Only Scan using idx_client_name_phone on public.client as cli... **Index Cond**: (client.name = ANY ('{Иван,Мария,Алексей,Елена}'::text[]))	0.084 ms	0.084 ms	↑ 1	4	4	1

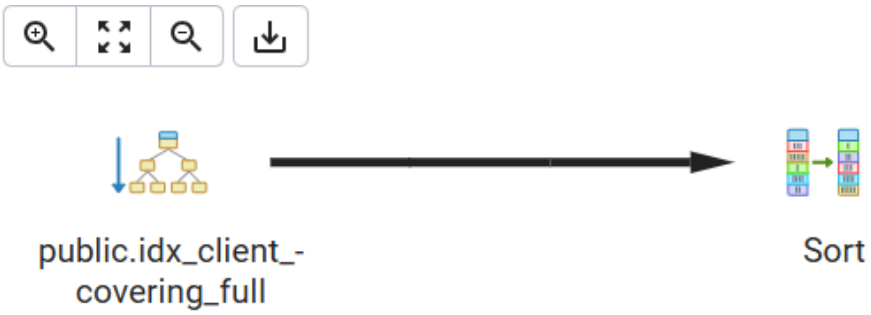
3.3 Частичный Hash индекс

```
SELECT Name, LastName, NumberPurchase,
  CASE
    WHEN NumberPurchase BETWEEN 5 AND 15 THEN 'Постоянный клиент'
      WHEN NumberPurchase BETWEEN 16 AND 25 THEN 'VIP-клиент'
      ELSE 'Супер-VIP'
  END AS ClientCategory
FROM Client
ORDER BY NumberPurchase DESC;
```



	#	Node	Timings		Rows			Loops
			Exclusive	Inclusive	Rows X	Actual	Plan	
	1.	→ Sort (cost=22.93..23.05 rows=50 width=59) (actual=0.036..0.038 ro...	0.015 ms	0.038 ms	↑ 1	50	50	1
	2.	→ Index Only Scan using idx_client_covering_full on public.client a...	0.023 ms	0.023 ms	↑ 1	50	50	1

CREATE INDEX idx_client_regular_name_hash ON Client USING HASH(Name) WHERE NumberPurchase BETWEEN 10 AND 20;



#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Sort (cost=22.93..23.05 rows=50 width=59) (actual=0.036..0.038 ro...	0.015 ms	0.038 ms	↑ 1	50	50	1
2.	→ Index Only Scan using idx_client_covering_full on public.client a...	0.023 ms	0.023 ms	↑ 1	50	50	1